

Module 4

Camera, RGB-D and Coordinate Transform

Tài liệu đào tạo nội bộ của BA x AILAB@UEL

Thông tin	Nội dung
Phiên bản	v1.0 FULL
Thời lượng khuyến nghị	1.5 ngày học tập trung + 0.5 ngày làm mini-project
Đối tượng	Sinh viên năm 3 ngành CNTT/KHMT, đã học Module 1-3 hoặc có kiến thức Python cơ bản
Vai trò chính	Perception/Data Vision guild; Action Planning và Manipulation guild cần học để hiểu đầu vào pose
Sản phẩm cuối module	Camera config sheet, point cloud, pixel-to-3D unit tests, camera-to-base transform report, pose overlay, baseline demo report

Ghi chú: Tài liệu này không giả định sinh viên đã thành thạo robotics. Các khái niệm được giải thích theo hướng phục vụ trực tiếp cho 4 task HRC2026: desktop sorting, conveyor sorting, workpiece insertion, carton sealing.

Mục lục tóm tắt

1. Vị trí của Module 4 trong pipeline HRC2026
2. Chuẩn đầu ra và cấu trúc học 3 lớp
3. Lý thuyết tối thiểu: camera, RGB-D, calibration, coordinate transform
4. Isaac Sim camera/RGB-D trong bối cảnh baseline
5. Demo trên baseline HRC2026
6. Bài lab có output đo được
7. Mini-project cuối module và rubric
8. Troubleshooting checklist
9. Câu hỏi tự kiểm tra
10. Tài liệu tham khảo và video/tutorial

1. Vị trí của Module 4 trong pipeline HRC2026

Module 4 nối trực tiếp Module 3 về SE(3), frame và transform với các module perception/manipulation phía sau. Trong HRC2026, robot không thể gấp, chèn hay đóng hộp chỉ từ ảnh RGB đẹp; hệ thống cần chuyển quan sát 2D/3D thành tọa độ dùng được cho planner và controller.

Từ quan sát	Đến dữ liệu cần cho robot	Ví dụ trong HRC2026
RGB image	class_id, mask, bbox, keypoints	Phân biệt hai loại workpieces ở Task 1/2
Depth image	3D point, point cloud, object height/pose	Tính điểm gấp trên mặt bàn hoặc băng chuyền
Camera intrinsics	pixel + depth -> 3D point trong camera frame	Từ centroid vật thể trong ảnh tính vị trí 3D
Camera extrinsics	camera frame -> world/base frame	Từ pose vật thể trong camera chuyển sang robot base
Pose estimation	object pose hoặc slot pose	Căn vật để chèn vào foam ở Task 3
Camera stream + tracking	velocity, predicted intercept point	Dự đoán vị trí vật trên băng chuyền Task 2

Sau module này, sinh viên cần hiểu một nguyên tắc nền tảng: perception không chỉ là vẽ bounding box; perception phải xuất dữ liệu định lượng để planner/motion module sử dụng được.

2. Chuẩn đầu ra của Module 4

Mã	Chuẩn đầu ra
M4-CLO1	Giải thích được pinhole camera model, intrinsic matrix K, pixel coordinate, depth và point cloud.
M4-CLO2	Tính được tọa độ 3D trong camera frame từ pixel (u, v) và depth z.
M4-CLO3	Phân biệt được intrinsic, extrinsic, camera frame, world frame, robot base frame và object frame.
M4-CLO4	Tạo được point cloud từ RGB-D image và trích xuất thống kê cơ bản về point cloud.
M4-CLO5	Dùng transform chain để chuyển point/pose từ camera frame sang world/base frame.
M4-CLO6	Hiểu calibration, distortion và lý do cần kiểm tra alignment giữa RGB và depth.
M4-CLO7	Biết cách inspect camera/sensor trong Isaac Sim/baseline và xuất camera config sheet.
M4-CLO8	Tạo được baseline demo report với ảnh RGB, depth, overlay điểm 3D và log transform.
M4-CLO9	Thiết kế được lab output đo được: CSV/JSON/PLY/PNG/report thay vì chỉ mô tả bằng lời.

3. Cấu trúc học 3 lớp cho Module 4

Lớp	Nội dung	Output bắt buộc
Lớp A - Lý thuyết tối thiểu	Pinhole camera, K matrix, depth, RGB-D, point cloud, calibration, extrinsic, coordinate transform, PnP/ICP ở mức khái niệm.	Notebook lý thuyết + unit tests công thức pixel-to-3D.
Lớp B - Demo trên baseline	Tìm camera trong scene/config, đọc thông số camera, capture RGB/depth, kiểm tra frame, chuyển một pixel/point sang frame robot/base.	baseline_camera_demo.md, camera_config_sheet.csv, sample_rgb.png, sample_depth.png.
Lớp C - Lab đo được	Tạo point cloud, tính thống kê depth, kiểm tra transform chain, vẽ overlay 2D/3D, báo cáo sai số hoặc sanity checks.	point_cloud.ply, transform_chain_report.json, overlay_axes.png, pytest_report.txt.

4. Lý thuyết tối thiểu về camera

4.1. Ảnh số, pixel và hệ tọa độ ảnh

Ảnh RGB là ma trận $H \times W \times 3$. Mỗi pixel có tọa độ ảnh (u, v), trong đó u thường là chỉ số cột theo chiều ngang, v là chỉ số hàng theo chiều dọc. Gốc tọa độ ảnh thường nằm ở góc trên-trái. Với robotics, cần hết sức cẩn thận vì hệ tọa độ ảnh, camera frame, world frame và robot base frame không giống nhau.

Ký hiệu	Ý nghĩa	Ghi chú
u	tọa độ cột pixel	tăng từ trái sang phải
v	tọa độ hàng pixel	tăng từ trên xuống dưới
z hoặc depth	khoảng cách đo bởi depth camera	đơn vị có thể là m, mm hoặc scale tùy nguồn
H, W	chiều cao, chiều rộng ảnh	ví dụ 480×640 hoặc 720×1280
bbox	bounding box	thường là $x_{\min}, y_{\min}, x_{\max}, y_{\max}$ hoặc x, y, w, h
mask	vùng pixel thuộc vật thể	cần cho centroid/pose/point cloud của vật

4.2. Pinhole camera model

Pinhole model mô tả cách điểm 3D trong camera frame chiếu lên ảnh 2D. Nếu điểm 3D trong camera frame là $P_{\text{cam}} = (X, Y, Z)$, pixel tương ứng được tính gần đúng bởi:

$$u = f_x * X / Z + c_x$$

$$v = f_y * Y / Z + c_y$$

Trong đó f_x, f_y là tiêu cự theo đơn vị pixel; c_x, c_y là tọa độ principal point/optical center. Ma trận intrinsic K có dạng:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Công thức ngược, dùng nhiều nhất trong module này, là chuyển từ pixel + depth sang điểm 3D trong camera frame:

$$X = (u - c_x) * Z / f_x$$

$$Y = (v - c_y) * Z / f_y$$

$$Z = \text{depth}$$

4.3. Depth image và các dạng depth thường gặp

Depth image là ảnh một kênh, mỗi pixel chứa giá trị khoảng cách. Tuy nhiên cần kiểm tra kỹ định nghĩa của depth vì các hệ thống có thể trả về distance-to-image-plane, distance-to-camera-center, disparity hoặc depth đã scale.

Loại dữ liệu	Ý nghĩa	Rủi ro thường gặp
Depth in meters	Mỗi pixel là khoảng cách tính bằng mét	Dễ nhầm với millimeters
Depth in millimeters	Mỗi pixel là khoảng cách tính bằng mm	Nếu không chia 1000, point cloud phóng đại 1000 lần
Distance to image plane	Khoảng cách theo trục Z camera	Phù hợp công thức pinhole đơn giản
Euclidean distance	Khoảng cách từ camera center đến điểm	Không giống Z nếu điểm lệch xa tâm ảnh
Disparity	Giá trị liên quan tới nghịch đảo depth	Cần biết baseline/focal để đổi sang depth

4.4. RGB-D image và alignment

RGB-D gồm một ảnh màu RGB và một ảnh depth tương ứng. Để dùng trực tiếp pixel RGB làm pixel depth, hai ảnh phải cùng resolution và đã được đăng ký/aligned vào cùng camera frame. Nếu RGB và depth không aligned, centroid/mask từ RGB sẽ lấy nhầm depth.

- Kiểm tra shape RGB và depth có giống nhau không.

- Kiểm tra depth tại centroid vật thể có hợp lý so với mặt bàn/scene không.
- Không lấy depth tại một pixel duy nhất nếu mask nhiều; nên dùng median depth trong vùng mask nhỏ.
- Loại bỏ giá trị depth = 0, NaN, inf hoặc quá xa/quá gần.
- Luôn ghi rõ depth_scale trong report.

4.5. Distortion và calibration

Camera thật thường có lens distortion, chủ yếu radial distortion và tangential distortion. Calibration tìm intrinsic K, distortion coefficients và đôi khi extrinsic giữa camera với một frame tham chiếu. Trong Isaac Sim, camera lý tưởng hơn camera thật, nhưng vẫn cần hiểu calibration để chuẩn bị cho vòng robot thật và để chuyển thông số OpenCV sang Isaac/Omniverse khi cần.

Khái niệm	Ý nghĩa thực dụng
Intrinsic calibration	Tìm f_x, f_y, c_x, c_y và distortion coefficients của camera.
Extrinsic calibration	Tìm pose của camera so với robot/world/base.
Undistortion	Sửa ảnh để đường thẳng ngoài đời không bị cong trên ảnh.
Reprojection error	Sai số giữa điểm 2D quan sát và điểm 2D chiếu từ mô hình camera; dùng để đánh giá calibration.

4.6. Camera extrinsic và transform chain

Intrinsic giúp chuyển pixel + depth sang point trong camera frame. Extrinsic giúp chuyển point đó sang world frame hoặc robot base frame. Theo quy ước từ Module 3, T_{A_B} là transform đưa tọa độ của một point từ frame B sang frame A.

```
p_cam_h = [X, Y, Z, 1]^T
p_world_h = T_world_cam @ p_cam_h
p_base_h = T_base_world @ T_world_cam @ p_cam_h
```

Nếu đã có T_{base_cam} :

```
p_base_h = T_base_cam @ p_cam_h
```

Đây là đường dẫn nhận thức tối thiểu cho robot:

```
RGB/depth pixel -> 3D point in camera frame -> 3D point in world/base frame -> grasp/insert/place target
```

4.7. Point cloud từ RGB-D

Point cloud là tập hợp các điểm 3D, thường kèm màu RGB. Từ depth image và K, ta có thể back-project nhiều pixel thành nhiều điểm 3D. Point cloud giúp ước lượng mặt phẳng bàn, centroid 3D, orientation bằng PCA, hoặc align vật thể với CAD/mesh.

Ứng dụng point cloud	Ví dụ trong HRC2026
Estimate centroid 3D	Tính vị trí chi tiết trên bàn hoặc băng chuyền.
Plane segmentation	Tách mặt bàn/foam khỏi vật thể.
PCA orientation	Ước lượng hướng của workpiece khi hình dạng kéo dài.
ICP/template alignment	Căn mesh/slot với point cloud cho Task 3.
Outlier removal	Loại điểm depth nhiễu để tăng ổn định pose.

4.8. Pose estimation 2D-3D, PnP và ICP ở mức tối thiểu

Pose estimation là ước lượng vị trí và hướng của vật thể. Với object có keypoints 3D đã biết và ảnh 2D keypoints, có thể dùng PnP để tìm rotation/translation. Với point cloud hoặc mesh, có thể dùng ICP để tinh chỉnh pose giữa point cloud quan sát và template/CAD.

Phương pháp	Đầu vào	Đầu ra	Khi nên dùng
Centroid + median depth	mask/bbox + depth	vị trí 3D gần đúng	Task 1/2 ban đầu, khi chỉ cần gấp ổn định
PCA on point cloud	mask depth points	hướng chính của vật	vật có hình dạng kéo dài hoặc cần orientation
PnP	2D keypoints + 3D keypoints	R, t của	vật có

	+ K	object/camera	keypoints/marker/geometry rõ
ICP/template alignment	point cloud quan sát + mesh/template	pose tinh chỉnh	Task 3, slot/foam/cad alignment

5. Camera và RGB-D trong Isaac Sim/baseline

Trong Isaac Sim, camera thường là một USD Camera prim. Camera data được lấy thông qua render product và các annotator/extension. Với HRC2026, sinh viên cần biết cách tìm camera trong stage, đọc thông số camera, capture RGB/depth/segmentation và kiểm tra camera pose so với robot/world.

Thành phần Isaac Sim	Ý nghĩa cho Module 4	Output sinh viên cần lấy được
Camera prim	Đối tượng camera trong stage/USD; có transform, focal length, aperture, clipping range.	camera prim path, pose, resolution.
Render product	Nguồn sinh ảnh từ camera ở một resolution cụ thể.	RGB/depth output path hoặc array shape.
Annotator	Cơ chế lấy dữ liệu như RGB, depth, segmentation, bounding boxes.	sample_rgb.png, sample_depth.npy/png.
USD Stage	Cây scene chứa robot, camera, table, objects, bins, foam, carton.	camera_inventory.csv, scene_camera_notes.md.
Replicator	Tạo dữ liệu synthetic như RGB, bbox, segmentation, depth.	synthetic_sample_manifest.json nếu dùng.

5.1. Các câu hỏi cần trả lời khi inspect camera trong baseline

11. Camera nằm ở prim path nào trong stage?
12. Camera gắn với robot, world hay scene object nào?
13. Resolution RGB/depth là bao nhiêu?
14. Camera output là RGB, depth, segmentation, bbox hay nhiều loại cùng lúc?
15. Depth unit là mét, millimeters hay một scale khác?
16. Camera intrinsic K lấy trực tiếp được không? Nếu không, tính từ focal length/aperture/FOV như thế nào?
17. Extrinsic camera-to-world lấy từ USD transform hay từ API baseline?
18. World frame và robot base frame khác nhau thế nào trong scene?
19. Có nhiều camera không? Camera nào dùng cho policy, camera nào dùng cho perception debug?
20. Dữ liệu camera có được ghi vào LeRobotDataset hay chỉ dùng online?

5.2. Gợi ý naming convention cho output camera

```

outputs/module4/
  camera_config_sheet.csv
  camera_inventory.csv
  sample_rgb.png
  sample_depth.npy
  sample_depth_vis.png
  point_cloud.ply
  point_cloud_stats.json
  transform_chain_report.json
  overlay_pixel_to_3d.png
  baseline_camera_demo.md

```

6. Demo trên baseline HRC2026

Các demo dưới đây được thiết kế để sinh viên có thể chạy sau khi đã hoàn thành Module 1-3 và đã clone baseline/assets. Nếu môi trường chưa chạy được Isaac Sim, sinh viên vẫn có thể làm bản dry-run bằng synthetic images/depth arrays để kiểm tra công thức.

Demo 1 - Baseline camera inventory

Mục tiêu: tìm tất cả camera hoặc cấu hình camera trong baseline/assets/config. Demo này không yêu cầu hiểu toàn bộ Isaac Sim API; chỉ cần scan file USD/YAML/Python và ghi lại các path/tên camera có khả năng liên quan.

```
# Chạy ở thư mục repo baseline
mkdir -p outputs/module4
find . -type f \( -name "*.py" -o -name "*.yaml" -o -name "*.yml" -o -name "*.usd" \) | grep -Ei
"camera|cam|sensor|rgb|depth" > outputs/module4/camera_related_files.txt

# Tìm keyword trong code/config
grep -rniE "camera|rgb|depth|render_product|annotator|segmentation" . >
outputs/module4/camera_keyword_hits.txt || true
```

Output đo được: `camera\_related\_files.txt`, `camera\_keyword\_hits.txt`, và bảng `camera\_inventory.csv` do sinh viên tổng hợp thủ công hoặc bán tự động.

Demo 2 - Capture RGB/depth frame từ baseline hoặc Isaac Sim sample

Mục tiêu: sinh viên chứng minh rằng mình có thể lấy được ít nhất một frame RGB và một frame depth. Nếu baseline có script sẵn để chạy task hoặc teleoperation, sinh viên dùng script đó; nếu chưa rõ, dùng Isaac Sim camera sample trước rồi quay lại baseline sau.

```
# Gợi ý output sau khi chạy demo
outputs/module4/sample_rgb.png
outputs/module4/sample_depth.npy
outputs/module4/sample_depth_vis.png
outputs/module4/frame_capture_log.json
```

Trường trong frame_capture_log.json	Ý nghĩa
timestamp_or_step	simulation step hoặc thời điểm capture
camera_prim_path	đường dẫn camera trong stage
rgb_shape	ví dụ [480, 640, 3]
depth_shape	ví dụ [480, 640]
depth_min_m, depth_median_m, depth_max_m	sanity check depth
camera_pose_world	translation + quaternion hoặc matrix

Demo 3 - Pixel + depth sang point 3D

Mục tiêu: chọn 5 pixel trong ảnh, đọc depth, tính điểm 3D trong camera frame và ghi ra CSV. Dùng median depth quanh một cửa sổ nhỏ để giảm nhiễu.

```
import numpy as np

def pixel_to_camera(u, v, z, fx, fy, cx, cy):
    X = (u - cx) * z / fx
    Y = (v - cy) * z / fy
    Z = z
    return np.array([X, Y, Z], dtype=float)

def median_depth(depth, u, v, radius=2):
```

```

patch = depth[max(0, v-radius):v+radius+1, max(0, u-radius):u+radius+1]
valid = patch[np.isfinite(patch) & (patch > 0)]
if len(valid) == 0:
    return np.nan
return float(np.median(valid))

```

Demo 4 - Transform camera point sang world/base frame

Mục tiêu: lấy một point trong camera frame, dùng `T_world_cam` hoặc `T_base_cam` để chuyển sang frame khác. Sinh viên cần kiểm tra bằng sanity check: tọa độ Z/height có hợp lý so với mặt bàn không, điểm có nằm trong vùng workspace không.

```

import numpy as np

def transform_point(T_A_B, p_B):
    p_h = np.r_[p_B, 1.0]
    p_A_h = T_A_B @ p_h
    return p_A_h[:3]

# Ví dụ: p_cam -> p_base
p_base = transform_point(T_base_cam, p_cam)

```

Demo 5 - Overlay điểm 3D và trục pose lên ảnh

Mục tiêu: kiểm tra trực quan công thức bằng cách vẽ lại điểm hoặc trục pose đã tính lên ảnh. Nếu điểm 3D chiếu ngược về pixel không gần pixel ban đầu, có thể sai K, sai depth scale hoặc sai convention frame.

```

def camera_to_pixel(P_cam, fx, fy, cx, cy):
    X, Y, Z = P_cam
    u = fx * X / Z + cx
    v = fy * Y / Z + cy
    return np.array([u, v])

```

7. Bài lab có output đo được

Mỗi lab phải nộp sản phẩm cụ thể. Không chấp nhận chỉ nộp screenshot hoặc giải thích miệng. Mục tiêu là giúp Evaluation guild có thể kiểm tra tự động hoặc bán tự động.

Lab 4.1 - Camera config sheet

Mục	Yêu cầu
Mục tiêu	Tìm và ghi lại thông số camera trong baseline/scene.
Input	Baseline repo, assets/config, hoặc một Isaac Sim scene có camera.
Output	camera_config_sheet.csv và camera_inventory.md.
Cột bắt buộc	camera_id, prim_path, resolution_w, resolution_h, fx, fy, cx, cy, depth_unit, frame_parent, source_file, notes.
Đạt	Có ít nhất 1 camera hợp lệ, có resolution và tối thiểu 4 intrinsic values hoặc ghi rõ cách tính/thiếu.

Lab 4.2 - Pixel-to-camera unit tests

Mục	Yêu cầu
Mục tiêu	Viết hàm pixel_to_camera và camera_to_pixel, kiểm tra round-trip.
Input	Một bộ K giả lập hoặc lấy từ camera_config_sheet.csv.
Output	pytest_report.txt, pixel_to_camera_report.csv.
Metric	Round-trip pixel error trung bình < 1e-6 với depth hợp lệ trong test synthetic.
Đạt	Ít nhất 10 test cases, có cả pixel ở center, corner, random và invalid depth.


```
# Gợi ý test case
K = dict(fx=600.0, fy=600.0, cx=320.0, cy=240.0)
for u, v, z in [(320,240,1.0), (0,0,1.5), (640,480,2.0), (123,345,0.8)]:
    P = pixel_to_camera(u, v, z, **K)
    uv = camera_to_pixel(P, **K)
    assert np.linalg.norm(uv - np.array([u, v])) < 1e-6
```

Lab 4.3 - RGB-D to point cloud

Mục	Yêu cầu
Mục tiêu	Tạo point cloud từ RGB-D và lưu file PLY.
Input	sample_rgb.png + sample_depth.npy/png hoặc synthetic depth array.
Output	point_cloud.ply, point_cloud_stats.json, point_cloud_preview.png.
Metric	Số điểm hợp lệ, min/max/median Z, bounding box point cloud.
Đạt	Point cloud có > 1000 điểm hợp lệ hoặc báo rõ lý do nếu dùng sample nhỏ; preview không bị lộn trục nghiêm trọng.

```
# Pseudocode với Open3D
import open3d as o3d
import numpy as np

# depth_m: numpy array HxW, đơn vị mét
# rgb: numpy array HxWx3
# tạo RGBDImage hoặc tự back-project nếu cần kiểm soát scale
# lưu: o3d.io.write_point_cloud('point_cloud.ply', pcd)
```

Lab 4.4 - Camera-to-base transform chain

Mục	Yêu cầu
Mục tiêu	Chuyển 5 điểm từ camera frame sang world/base frame.
Input	T_world_cam hoặc T_base_cam giả lập/baseline; 5 point từ Lab 4.2/4.3.
Output	transform_chain_report.json, transformed_points.csv.
Metric	Kiểm tra T inverse, determinant của R gần 1, round-trip error nhỏ.
Đạt	Mọi transform 4x4 hợp lệ; round-trip p_cam -> p_base -> p_cam sai số < 1e-6 trong test synthetic.

Lab 4.5 - Object mask to 3D centroid

Mục	Yêu cầu
Mục tiêu	Từ mask/bbox của vật, tính centroid 2D, median depth, centroid 3D và overlay lên ảnh.
Input	RGB, depth, mask tự tạo hoặc bbox thủ công.
Output	object_3d_centroids.csv, overlay_centroid.png.
Metric	Có centroid_2d, median_depth_m, centroid_cam_xyz, centroid_base_xyz nếu có T_base_cam.
Đạt	Ít nhất 3 object/mask hoặc 3 vùng test; không dùng pixel depth đơn lẻ nếu có mask.

Lab 4.6 - Pose estimation mini-demo: PnP hoặc PCA/ICP

Mục	Yêu cầu
Mục tiêu	Chạy một phương pháp pose estimation đơn giản.
Phương án A	PnP với 4-8 keypoints 3D/2D synthetic; xuất rvec, tvec, reprojection error.
Phương án B	PCA trên point cloud mask; xuất principal axis và orientation angle.
Phương án C	ICP giữa point cloud synthetic và template; xuất fitness/RMSE.
Output	pose_estimator_report.csv, pose_overlay.png hoặc icp_alignment_report.json.
Đạt	Có metric định lượng: reprojection error, PCA consistency hoặc ICP RMSE/fitness.

8. Mini-project cuối module: Camera-to-Base Perception Readiness

Mini-project này kiểm tra xem sinh viên đã đủ sẵn sàng để bước sang Module 5 - Perception for Industrial Manipulation hay chưa. Sản phẩm phải chạy được trên dữ liệu sample/synthetic và nếu có môi trường thì chạy được trên baseline/Isaac Sim.

Thành phần nộp	Mô tả
README.md	Mô tả cách chạy, môi trường, input/output, hạn chế.
camera_config_sheet.csv	Thông số camera và nguồn lấy thông số.
sample_rgb.png, sample_depth_vis.png	Ảnh demo lấy từ baseline/Isaac Sim hoặc synthetic.
pixel_to_camera.py	Hàm chuyển pixel-depth sang 3D camera frame.
transforms.py	Hàm transform point/pose giữa frame.
point_cloud.ply	Point cloud từ RGB-D.
overlay_centroid.png	Ảnh có vẽ centroid/object point/axis.
transform_chain_report.json	Kiểm tra transform, inverse, determinant, round-trip error.
pytest_report.txt	Kết quả test công thức.
baseline_camera_demo.md	Ghi chú demo trên baseline: camera path, resolution, depth stats, lỗi gặp phải.

8.1. Rubric chấm mini-project 100 điểm

Nhóm tiêu chí	Điểm	Mô tả chấm
Camera understanding	15	Giải thích đúng intrinsic/extrinsic/depth unit; camera_config_sheet đầy đủ.
Pixel-to-3D correctness	15	Công thức đúng, unit tests pass, xử lý invalid depth.
Coordinate transform	15	T_base_cam/T_world_cam đúng convention, inverse/round-trip check rõ.
RGB-D/point cloud	15	Tạo point cloud hợp lệ, có stats và preview.
Baseline demo	15	Có demo/scan baseline, capture hoặc phân tích camera config; ghi lỗi và cách xử lý.
Pose/centroid overlay	10	Có overlay trực quan và CSV định lượng.
Reproducibility	10	README rõ, lệnh chạy một lần, output đúng cấu trúc.
Engineering discipline	5	Code sạch, đặt tên file rõ, log đầy đủ, không hard-code mơ hồ.

8.2. Ngưỡng đánh giá

Mức	Điểm	Diễn giải
Excellent	85-100	Có thể tham gia Perception guild hoặc hỗ trợ Manipulation với pose/transform.
Good	70-84	Nắm lỗi RGB-D/transform, cần thêm thực hành baseline.
Pass	60-69	Có thể đi tiếp nhưng cần mentor review phần frame/depth scale.
Retry	<60	Chưa đủ sẵn sàng cho Module 5; cần làm lại Lab 4.2-4.4.

9. Troubleshooting checklist

Hiện tượng	Nguyên nhân thường gặp	Cách kiểm tra/sửa
Point cloud quá lớn/quá xa	Nhầm depth mm thành m hoặc ngược lại.	In min/median/max depth; kiểm tra depth_scale.
Point cloud bị lộn ngược	Khác convention giữa image frame, camera frame, Open3D/Isaac.	Vẽ 3 trục camera; kiểm tra trục Y/Z.
Centroid 3D nằm dưới bàn hoặc sau camera	Sai depth tại mask, sai invalid depth, sai T.	Dùng median depth trong mask; kiểm tra Z > 0; test transform inverse.

Overlay điểm không trùng pixel ban đầu	Sai fx/fy/cx/cy hoặc sai công thức project/back-project.	Chạy round-trip <code>camera_to_pixel(pixel_to_camera())</code> .
Depth toàn 0/NaN	Camera chưa render, annotator chưa attach, sai render product.	Chạy thêm vài frame; kiểm tra shape và log render product.
RGB và depth lệch nhau	Không aligned hoặc khác resolution.	Kiểm tra shape; resize/alignment; không dùng mask RGB trực tiếp nếu depth không aligned.
Transform đúng số nhưng robot gấp lệch	Sai quy ước T_{A_B}/T_{B_A} hoặc frame parent.	Viết rõ $p_A = T_{A_B} @ p_B$; test với điểm gốc/trục đơn vị.
Kết quả thay đổi giữa các lần chạy	Seed/randomization/render timing chưa cố định.	Ghi seed, step, timestamp; capture sau khi scene stable.

10. Câu hỏi tự kiểm tra

21. Sự khác nhau giữa intrinsic và extrinsic là gì?
22. Vì sao cần biết fx, fy, cx, cy để chuyển pixel sang 3D?
23. Nếu depth image dùng đơn vị millimeters nhưng code giả định meters, lỗi sẽ biểu hiện thế nào?
24. Tại sao không nên lấy depth tại một pixel duy nhất ở centroid?
25. RGB và depth không cùng resolution thì pipeline sẽ sai ở đâu?
26. T_{A_B} có nghĩa là gì? $p_A = T_{A_B} @ p_B$ hay $p_B = T_{A_B} @ p_A$?
27. Point cloud có thể giúp ước lượng orientation của vật thể như thế nào?
28. PnP cần những input nào?
29. ICP khác PnP ở đâu?
30. Trong Task 2 băng chuyền, camera/RGB-D hỗ trợ tracking như thế nào?
31. Trong Task 3 insertion, vì sao pose/slot frame quan trọng hơn classification accuracy?
32. Ba output định lượng nào chứng minh lab camera/transform đã chạy đúng?

11. Tài liệu tham khảo và video/tutorial

Ưu tiên nguồn mở, tài liệu chính thức từ nhà cung cấp và GitHub liên quan. Sinh viên nên đọc theo thứ tự: Isaac Sim camera/sensor -> OpenCV calibration/PnP -> Open3D RGB-D/point cloud/ICP -> baseline/assets HRC2026.

Tài liệu	Link	Dùng để làm gì
UBTECH HRC2026 Baseline GitHub	https://github.com/UBTECH-Robot/GlobalHumanoidRobotChallenge_2026_Baseline	Repo baseline chính thức: simulation, data collection, training, inference, evaluation.
UBTECH challenge2026 assets	https://huggingface.co/UBTECH-Robotics/challenge2026_assets	Assets Walker S2 và scene/task components.
UBTECH challenge2026 dataset	https://huggingface.co/datasets/UBTECH-Robotics/challenge2026_dataset	Dataset LeRobot v2.1 để học cấu trúc dữ liệu camera/video/state/action.
NVIDIA Isaac Sim Sensors	https://docs.isaacsim.omniverse.nvidia.com/5.1.0/sensors/index.html	Tổng quan sensor simulation, camera, depth, RTX, physics-based sensors.
NVIDIA Isaac Sim Camera Sensors	https://docs.isaacsim.omniverse.nvidia.com/4.5.0/sensors/isaacsim_sensors_camera.html	Camera prim, render product, calibrated camera sensors.

NVIDIA Isaac Sim Depth Sensors	https://docs.isaacsim.omniverse.nvidia.com/5.1.0/sensors/isaacsim_sensors_camera_depth.html	Depth camera, depth annotators, single-view depth sensor examples.
NVIDIA Isaac Sim Add Camera and Sensors Tutorial	https://docs.isaacsim.omniverse.nvidia.com/4.5.0/gui/tutorial_gui_camera_sensors.html	Tutorial GUI thêm camera/sensor và attach vào robot.
NVIDIA Getting Started With Isaac Sim - Adding Sensors	https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/building-your-first-robot-in-isaac-sim/03-adding-sensors.html	Bài học có video hướng dẫn thêm RGB sensor, xem camera view và dùng dual viewports.
Isaac Sim Replicator Object / Synthetic Data	https://docs.isaacsim.omniverse.nvidia.com/latest/action_and_event_data_generation/tutorial_replicator_object.html	Sinh dữ liệu synthetic RGB, 2D/3D boxes, segmentation masks.
OpenCV Camera Calibration	https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html	Intrinsic/extrinsic, distortion, camera matrix, undistortion.
OpenCV Camera Calibration with OpenCV	https://docs.opencv.org/4.x/d4/d94/tutorial_camera_calibration.html	Lý thuyết calibration và distortion coefficients.
OpenCV Pose Estimation / PnP	https://docs.opencv.org/4.x/d7/d53/tutorial_py_pose.html	Ước lượng pose từ camera matrix, distortion coefficients và keypoints.
Open3D RGBD images	https://www.open3d.org/docs/0.10.0/tutorial/Basic/rgbd_image.html	Tạo RGBDImage và point cloud từ color/depth.
Open3D Point Cloud Tutorial	https://open3d.org/docs/release/tutorial/geometry/pointcloud.html	Đọc, hiển thị, xử lý point cloud.
Open3D ICP Registration	https://www.open3d.org/docs/latest/tutorial/Basic/icp_registration.html	Align point clouds bằng ICP.
ROS 2 tf2 Concepts	https://docs.ros.org/en/humble/Concepts/Intermediate/About-Tf2.html	Tư duy quản lý nhiều coordinate frames theo thời gian.
Modern Robotics - Rigid-Body Motions	https://modernrobotics.northwestern.edu/chapters/chapter3/	Nền tảng SE(3), frame và transform, liên kết trực tiếp Module 3.

Phụ lục A. Code skeleton đề xuất cho sinh viên

module4_camera_rgbd/

README.md

```

src/
  camera_math.py
  depth_utils.py
  transforms.py
  point_cloud_utils.py
  overlay_utils.py
tests/
  test_camera_math.py
  test_transforms.py
outputs/
  camera_config_sheet.csv
  pixel_to_camera_report.csv
  transform_chain_report.json
  point_cloud.ply
  overlay_centroid.png
  pytest_report.txt

```

A.1. camera_math.py tối thiểu

```

import numpy as np

def pixel_to_camera(u, v, z, fx, fy, cx, cy):
    if not np.isfinite(z) or z <= 0:
        raise ValueError('Depth must be positive and finite')
    X = (u - cx) * z / fx
    Y = (v - cy) * z / fy
    Z = z
    return np.array([X, Y, Z], dtype=float)

def camera_to_pixel(P, fx, fy, cx, cy):
    X, Y, Z = P
    if not np.isfinite(Z) or Z <= 0:
        raise ValueError('Z must be positive and finite')
    u = fx * X / Z + cx
    v = fy * Y / Z + cy
    return np.array([u, v], dtype=float)

def make_K(fx, fy, cx, cy):
    return np.array([[fx, 0.0, cx], [0.0, fy, cy], [0.0, 0.0, 1.0]], dtype=float)

```

A.2. transforms.py tối thiểu

```

import numpy as np

def check_transform(T, atol=1e-6):
    T = np.asarray(T, dtype=float)
    assert T.shape == (4, 4)
    R = T[:3, :3]
    det_R = np.linalg.det(R)
    ortho_err = np.linalg.norm(R.T @ R - np.eye(3))
    return {'det_R': float(det_R), 'ortho_err': float(ortho_err),
            'is_valid': abs(det_R - 1.0) < 1e-3 and ortho_err < 1e-3}

```

```
def transform_point(T_A_B, p_B):
    p_B_h = np.r_[np.asarray(p_B, dtype=float), 1.0]
    p_A_h = np.asarray(T_A_B, dtype=float) @ p_B_h
    return p_A_h[:3]

def inverse_transform(T_A_B):
    return np.linalg.inv(np.asarray(T_A_B, dtype=float))
```

A.3. depth_utils.py tối thiểu

```
import numpy as np

def valid_depth_mask(depth, min_depth=0.05, max_depth=5.0):
    depth = np.asarray(depth, dtype=float)
    return np.isfinite(depth) & (depth >= min_depth) & (depth <= max_depth)

def median_depth_in_mask(depth, mask):
    valid = valid_depth_mask(depth) & (mask.astype(bool))
    values = depth[valid]
    if values.size == 0:
        return np.nan
    return float(np.median(values))
```

Phụ lục B. Checklist nộp Module 4

File/thư mục	Bắt buộc?	Tự kiểm tra
README.md	Có	Lệnh chạy rõ, không phụ thuộc đường dẫn cá nhân.
camera_config_sheet.csv	Có	Có ít nhất 1 camera, resolution, K hoặc ghi rõ chưa lấy được K.
pytest_report.txt	Có	Lab 4.2 và 4.4 pass test synthetic.
sample_rgb.png	Có	Ảnh mở được.
sample_depth_vis.png	Có	Depth visualization không toàn đen/trắng.
point_cloud.ply	Có	Open3D đọc được, số điểm hợp lệ > 0.
point_cloud_stats.json	Có	Có min/median/max depth và bounding box.
transform_chain_report.json	Có	Có det_R, ortho_err, round_trip_error.
overlay_centroid.png	Có	Centroid/axis overlay nằm đúng vùng vật thể.
baseline_camera_demo.md	Có	Ghi camera path, input/output, lỗi gặp phải.